UyJoy V2 — Launch Roadmap

Step-by-step plan for codex. Run one step at a time, in order. Each step is self-contained — Codex doesn't need the whole document, just the step.

How to use this document

Send Codex one step at a time. Copy the **Codex prompt** from the step into your chat with Codex. When Codex reports it's done, verify the **Done when** checklist before moving on. If a step depends on a previous step, that's noted.

What's already done (from the first batch)

• PII leak fixed on /projects/[id] and /projects/[id]/explore (publicProjectSelect).
• POST /api/leads hardened: phone normalization, honeypot, per-IP 5/min, source allowlist.
• AI routes hardened: path-traversal blocked, only /uploads/* local or HTTPS Cloudinary external.
• Upload route: sharp magic-byte check, MIME spoof block, EXIF strip, 20/5min rate limit.
• Security headers in next.config.mjs (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy).
• requireAdmin() helper + server-side guards on admin server pages.
• CSV export in LeadsClient now quoted/escaped, formula-prefix neutralized.
• TypeScript clean, lint clean, build clean.

What this document covers

Stages 1–7 take the project from 'first-batch hardened' to 'paying client ready'. Stage 1 is mandatory for any launch (compliance + observability). Stage 2 is mandatory before your admin team can trust the dashboard. Stage 3 is mandatory only if you're selling to more than one developer. Stages 4–6 are quality and conversion. Stage 7 is the final smoke test.

Table of contents

Stage 1 — Compliance, legal, and observability

Estimated time: about 1 day, can be split across two evenings. These three are non-negotiable for a paying real-estate client. Sentry catches the first production bug you don't know exists. Privacy/Terms makes the lead form legally collectable under Uzbek PD Law 547-IV. Turnstile catches the bot wave that arrives within hours of the site appearing in Google.

Step 1.1 — Add Sentry error monitoring

Why: There is currently zero runtime visibility. A 500 in production today goes unnoticed.

Files to touch:

```
package.json
src/instrumentation.ts (new)
sentry.client.config.ts (new)
sentry.server.config.ts (new)
sentry.edge.config.ts (new)
.env.example
.env
```

What to do:

• Install @sentry/nextjs with the Sentry wizard (npx @sentry/wizard@latest -i nextjs).

• Add NEXT_PUBLIC_SENTRY_DSN and SENTRY_AUTH_TOKEN to .env.example.

• In sentry.client.config.ts set tracesSampleRate: 0.1 and replaysSessionSampleRate: 0.

• Verify the wizard injected withSentryConfig() in next.config.mjs without removing the existing headers/rewrites.

• Add a temporary 'throw new Error("sentry test")' in any admin page, hit it once, confirm event lands in Sentry, then remove.

Done when:

• next build still passes.

• A test error appears in the Sentry dashboard.

• Existing headers() and rewrites() blocks are intact in next.config.mjs.

Codex prompt: *Add Sentry to this Next.js 14 app using the official wizard. Keep the existing security headers and PostHog rewrites in next.config.mjs intact. Sample tracing at 10%, disable session replays for now. Add a quick test error to verify capture, then remove it. Update .env.example with NEXT_PUBLIC_SENTRY_DSN and SENTRY_AUTH_TOKEN keys.*

Step 1.2 — Add Privacy Policy and Terms pages

Why: Capturing names and phone numbers in Uzbekistan requires a published privacy notice (PD Law 547-IV). The lead form must link to it.

Files to touch:

```
src/app/privacy/page.tsx (new)
src/app/terms/page.tsx (new)
src/components/Footer.tsx
src/components/ContactForm.tsx
src/components/ApartmentLeadModal.tsx
src/components/FloatingContact.tsx
messages/uz.json, messages/ru.json, messages/en.json
```

What to do:

• Create /privacy and /terms as server components, styled to match the warm-clay brand (reuse Navbar/Footer).

• Content sections needed in Privacy: what we collect (name, phone, unit interest), how we store it (Postgres on Vercel + Cloudinary for uploads), who has access (sales team + Anthropic-style data sub-processors), retention period (24 months from last contact), how to request deletion (email).

• Content sections needed in Terms: scope of service, no purchase guarantee, info accuracy disclaimer, dispute jurisdiction.

• Add 'By submitting, you agree to our Privacy Policy' below every submit button on lead forms, with /privacy linked.

• Add Privacy and Terms links in Footer column 'Information'.

• Add Uzbek, Russian, English translation keys for the disclaimer line.

Done when:

• /privacy and /terms render in all three locales.

• Every lead-submit button has a visible Privacy link next to it.

• Footer shows the two links.

Codex prompt: *Create /privacy and /terms pages as server components matching the brand (warm clay #c66348 / cream #f4efe7, Cormorant Garamond headings, Plus Jakarta body). Use Uzbek as primary locale, Russian and English versions following the same content. Required Privacy sections: data collected (name, phone, unit interest), storage (Postgres on Vercel, images on Cloudinary), retention (24 months from last contact), how to request deletion. Required Terms sections: scope of service, no purchase guarantee, info accuracy disclaimer, jurisdiction. Then add 'By submitting, you agree to our Privacy Policy' next to every lead-submit button (ContactForm, ApartmentLeadModal, FloatingContact help chooser) with /privacy linked. Add the new strings to all three messages/*.json files. Add /privacy and /terms to the Footer 'Information' column.*

Step 1.3 — Add Cloudflare Turnstile captcha to the lead form

Why: Free, no-friction bot protection. Pairs with the rate limit already in place.

Files to touch:

```
src/components/ContactForm.tsx
src/components/ApartmentLeadModal.tsx
src/components/FloatingContact.tsx
src/app/api/leads/route.ts
src/lib/turnstile.ts (new)
.env.example, .env
```

What to do:

• Add NEXT_PUBLIC_TURNSTILE_SITE_KEY and TURNSTILE_SECRET_KEY env vars.

• Mount the Turnstile widget inside every lead form via the official @marsidev/react-turnstile package (or raw script).

• On submit, include the turnstile token in the POST body as cfTurnstileToken.

• Add src/lib/turnstile.ts that POSTs to https://challenges.cloudflare.com/turnstile/v0/siteverify and returns boolean.

• In /api/leads POST: if env vars are set, require + verify the token; if not set, log a warning but accept (so dev still works).

Done when:

• Submitting a lead with the widget unchecked fails with 400.

• Submitting a lead with a valid Turnstile token succeeds.

• If env vars are missing, the form still works in dev and a warning is logged.

Codex prompt: *Add Cloudflare Turnstile to all three lead forms (ContactForm, ApartmentLeadModal, FloatingContact). Use @marsidev/react-turnstile. Add NEXT_PUBLIC_TURNSTILE_SITE_KEY and TURNSTILE_SECRET_KEY to .env.example. Create src/lib/turnstile.ts with verifyTurnstileToken(token, ip) that calls Cloudflare's siteverify endpoint. In /api/leads POST, if env keys are present, require body.cfTurnstileToken and verify it; if env keys are absent, log a warning and accept (dev-mode bypass). Send the remote IP to siteverify.*

Step 1.4 — Tie the FloatingContact green dot to actual sales hours

Why: Today the persistent green 'online' glow lies when no one is monitoring. Cheap trust win.

Files to touch:

```
src/components/FloatingContact.tsx
src/lib/sales-hours.ts (new)
messages/*.json (optional)
```

What to do:

• Create src/lib/sales-hours.ts with isWithinSalesHours(now = new Date()) returning boolean.

• Sales hours: Mon–Sat 09:00–19:00 Asia/Tashkent. Sunday closed.

• In FloatingContact, if outside hours: replace the green dot with a small Clock icon and show 'Replies in <1 business hour' tooltip on hover.

Done when:

• Visiting the site on Sunday shows the clock icon, not the green dot.

• Visiting at 10:00 Monday shows the green dot.

Codex prompt: *Create src/lib/sales-hours.ts exporting isWithinSalesHours() that returns true Mon–Sat 09:00–19:00 in Asia/Tashkent timezone. In src/components/FloatingContact.tsx, render the green 'online' dot only when isWithinSalesHours() is true; otherwise render a Clock icon (from lucide-react) at the same position with title='Replies in <1 business hour'.*

Stage 2 — Data integrity and stale-UI fixes

Estimated time: about 1 day. These four close the gap between admin actions and what the public sees, prevent admin foot-guns, and add the indexes you'll need before you have 10k units.

Step 2.1 — revalidateTag on every content mutation

Why: Today only units and FAQs invalidate the public cache. Renaming a building or uploading an aerial view leaves the public site stale for up to 60 seconds.

Files to touch:

```
src/lib/cache.ts (new)
src/app/api/projects/[id]/route.ts
src/app/api/buildings/route.ts
src/app/api/buildings/[id]/route.ts
src/app/api/floors/route.ts
src/app/api/floors/[id]/route.ts
src/app/api/floors/[id]/copy-to-all/route.ts
src/app/api/hero-images/route.ts
src/app/api/hero-images/[id]/route.ts
src/lib/cached-queries.ts
```

What to do:

• Create src/lib/cache.ts exporting invalidateProject() that calls revalidateTag('project') and revalidateTag('hero-images').

• Call invalidateProject() at the end of every POST/PUT/DELETE in the listed routes.

• Confirm getCachedHeroImages already has tags: ['hero-images'] (it does).

Done when:

• Renaming a building in admin and refreshing the homepage within 2 seconds shows the new name.

• Deleting a floor reflects on /kvartiralar immediately.

• Uploading a new hero image immediately replaces the old one on /.

Codex prompt: *Create src/lib/cache.ts that exports invalidateProject() which calls revalidateTag('project') and revalidateTag('hero-images'). Then call invalidateProject() after every successful mutation in: /api/projects/[id] (PUT), /api/buildings (POST), /api/buildings/[id] (PUT, DELETE), /api/floors (POST), /api/floors/[id] (PUT, DELETE), /api/floors/[id]/copy-to-all (POST), /api/hero-images (POST), /api/hero-images/[id] (DELETE). Keep the existing revalidateTag('project') calls in /api/units routes; just add invalidateProject() instead so behavior is consistent.*

Step 2.2 — Make copy-to-all transactional and PII-safe

Why: Today this endpoint deletes all units on all other floors in a non-transactional loop, resets reserved/sold to available (loses customer PII), and renames source-floor units as a side effect.

Files to touch:

```
src/app/api/floors/[id]/copy-to-all/route.ts
src/app/portal/management-x7k9/projects/[projectId]/buildings/[buildingId]/floors/[floorId]/editor/page.tsx
```

What to do:

• Wrap the entire copy in one prisma.$transaction([]) array (interactive transaction, not the sequential helper).

• For each target floor: if a unit at the same position is reserved or sold, skip it and keep its customerName/Phone/Notes/status. Only overwrite available units.

• Stop renaming source-floor units. The source floor is the source of truth; don't mutate it.

• In the admin editor, change the confirm() to a typed-confirm modal: user must type the floor number to enable the button. Show 'This will replace unit layouts on floors X, Y, Z. Reserved/sold units will be preserved.'

Done when:

• Running copy-to-all leaves source-floor unit numbers unchanged.

• A reserved unit on a target floor keeps its customerPhone after copy-to-all.

• If any single step fails, no floor is left half-updated.

• Admin must type the floor number before the button enables.

Codex prompt: *Refactor /api/floors/[id]/copy-to-all to: (1) wrap everything in a single prisma.$transaction([...]) array, (2) stop renaming source-floor units (delete that block), (3) for each target floor, preserve units whose status is 'reserved' or 'sold' along with their customerName/customerPhone/customerNotes — only overwrite 'available' units. Then update the floor editor page (src/app/portal/management-x7k9/projects/[projectId]/buildings/[buildingId]/floors/[floorId]/editor/page.tsx) to replace the window.confirm() with a typed-confirm modal: the admin must type the floor number into a text input before the destructive button enables, and the modal must list the target floor numbers and the count of reserved/sold units that will be preserved.*

Step 2.3 — Add Lead snapshot fields to survive deletes

Why: Lead.unitId is a bare string with no FK. When a unit is deleted, lead reports break.

Files to touch:

```
prisma/schema.prisma
src/app/api/leads/route.ts
src/app/portal/management-x7k9/leads/LeadsClient.tsx
```

What to do:

• Add to Lead: unitNumberSnapshot String?, unitAreaSnapshot Float?, unitRoomsSnapshot Int?, unitPriceSnapshot Float?, buildingNameSnapshot String?, floorNumberSnapshot Int?.

• In POST /api/leads, when unitId is provided, look up the unit and populate the snapshot fields.

• In LeadsClient and the dashboard, display the snapshot fields if the live unit no longer exists.

• Migration: npx prisma migrate dev --name lead-snapshots.

Done when:

• Submitting a lead for unit X populates snapshot fields.

• Deleting unit X afterwards leaves the lead intact with snapshot data visible in admin.

Codex prompt: *Add to the Lead model in prisma/schema.prisma the snapshot fields: unitNumberSnapshot String?, unitAreaSnapshot Float?, unitRoomsSnapshot Int?, unitPriceSnapshot Float?, buildingNameSnapshot String?, floorNumberSnapshot Int?. Run npx prisma migrate dev --name lead-snapshots. Then in POST /api/leads, when body.unitId is provided, query that unit including floor and building and populate the snapshot fields on creation. Update LeadsClient to render the snapshot fields when present, falling back to the live unitNumber. Don't break the existing UnitNumber column — display 'unitNumber (snapshot)' if the unit lookup at fetch time is null.*

Step 2.4 — Add the missing database indexes

Why: Every hot query (unit list, lead pagination, status filter) is currently a full table scan beyond a few hundred rows.

Files to touch:

```
prisma/schema.prisma
```

What to do:

• Add @@index([floorId]) to Unit.
• Add @@index([status]) to Unit.
• Add @@index([buildingId]) to Floor.
• Add @@index([projectId]) to Building.
• Add @@index([projectId]) to Lead.
• Add @@index([unitId]) to Lead.
• Add @@index([status, createdAt(sort: Desc)]) to Lead.
• Run npx prisma migrate dev --name add-indexes.

Done when:

• Migration applies cleanly.
• EXPLAIN ANALYZE on prisma.lead.findMany ordered by createdAt desc shows index scan, not seq scan.

Codex prompt: *Add these indexes to prisma/schema.prisma using @@index: Unit ([floorId]) and ([status]); Floor ([buildingId]); Building ([projectId]); Lead ([projectId]), ([unitId]), and ([status, createdAt(sort: Desc)]). Then run npx prisma migrate dev --name add-indexes.*

Stage 3 — Multi-tenancy (only if selling to more than one client)

Estimated time: about 2 days. Skip this stage if you're launching for one specific client (e.g., Navruz Residence) and will handle future clients with separate Vercel deployments. Do this stage if the product is sold as a SaaS to multiple developers from a single deployment. The two big blockers are findFirst() everywhere and hardcoded contact strings.

Step 3.1 — Add a ProjectSettings table

Why: All currently-hardcoded contact strings (phone, Telegram, address, expected year, master-plan filename, sales hours) move into the DB so each project carries its own.

Files to touch:

```
prisma/schema.prisma
src/lib/tenant.ts (new)
src/lib/cached-queries.ts
```

What to do:

• Add to Project: phoneNumber String?, telegramUrl String?, instagramUrl String?, salesOfficeAddress String?, salesHoursStart String? (HH:mm), salesHoursEnd String?, salesDaysJson String? ('1,2,3,4,5,6'), expectedYear Int?, masterPlanImage String?, brandLogo String?.

• Migration: npx prisma migrate dev --name project-settings.

• Create src/lib/tenant.ts with getCurrentTenant() that returns the current project (still findFirst for now — step 3.2 changes this) and exposes all the new fields.

• Update getCachedProject and friends to include the new fields.

Done when:

• Migration applies. All new fields are present on the Project model.

• getCurrentTenant() returns the project with all settings.

Codex prompt: *Add new fields to the Project model in prisma/schema.prisma: phoneNumber, telegramUrl, instagramUrl, salesOfficeAddress, salesHoursStart, salesHoursEnd, salesDaysJson, expectedYear (already exists, leave it), masterPlanImage, brandLogo. Run npx prisma migrate dev --name project-settings. Create src/lib/tenant.ts that exports getCurrentTenant() — for now it can still do prisma.project.findFirst() — returning the project including the new fields. Update src/lib/cached-queries.ts to include the new fields in publicProjectSelect.*

Step 3.2 — Resolve tenant by host or path

Why: Replace findFirst with a real tenant lookup so multiple projects can coexist.

Files to touch:

```
src/middleware.ts
src/lib/tenant.ts
prisma/schema.prisma
```

What to do:

• Add Project.slug String @unique and Project.domain String? @unique.

• Backfill the single existing project with slug='navruz' and (optionally) domain='navruz.uy-joy.com'.

• In middleware, read req.headers.get('host'). If it matches a known Project.domain, set x-tenant-id header. Otherwise use the default project.

• In getCurrentTenant(), read the x-tenant-id header (via headers() from next/headers) and findUnique by id.

• Update unstable_cache keys to include tenant id: ['project-with-units', tenantId].

Done when:

• Setting a custom domain to point to the Vercel deploy serves that project's content.

• navruz.uy-joy.com shows Navruz; any other domain shows whatever the default tenant is.

• Two projects in the DB can be served from one deployment without cache leaking between them.

Codex prompt: *Add Project.slug (String @unique) and Project.domain (String? @unique) to prisma/schema.prisma and migrate. Backfill the existing project with slug='navruz'. In src/middleware.ts, look up tenant by request host using prisma.project.findUnique({ where: { domain: host } }) — fall back to the first project if no match. Pass the tenant id forward via a request header 'x-tenant-id'. Refactor src/lib/tenant.ts to read that header via headers() from next/headers and prisma.project.findUnique({ where: { id } }). Update all unstable_cache calls in src/lib/cached-queries.ts to make the cache key include tenantId.*

Step 3.3 — Purge every hardcoded contact string

Why: Replace @wxusan, +998 77 041 12 22, 'Navruz Residence', PROJECT_EXPECTED_YEAR, navruz-residence-master.png with tenant settings reads.

Files to touch:

```
src/lib/project-copy.ts (delete after migration)
src/components/FloatingContact.tsx
src/components/Footer.tsx
src/components/NavbarClient.tsx
src/components/ContactForm.tsx
src/components/ProjectTopView.tsx
src/app/page.tsx
src/app/kvartiralar/page.tsx
src/app/vizual/page.tsx (will be deleted in stage 4 anyway)
src/app/projects/[projectId]/page.tsx (will be deleted)
src/app/projects/[projectId]/explore/page.tsx (will be deleted)
```

What to do:

• Every component using contact info reads from getCurrentTenant() instead.

• FloatingContact phone/telegram come from tenant.phoneNumber / tenant.telegramUrl. If null, hide the action button.

• Footer copyright + project name read from tenant.name.

• ContactForm phone link and telegram link come from tenant.

• ProjectTopView fallback image uses tenant.masterPlanImage; if null, show empty state instead of falling back to navruz file.

• PROJECT_EXPECTED_YEAR sourced from tenant.expectedYear.

• After this step, search the source for 'wxusan', '+998 77', 'navruz-residence' — should return zero matches in src/ outside of /public/uploads.

Done when:

• grep -rn 'wxusan' src/ returns nothing.

• grep -rn '+998 77 041' src/ returns nothing.

• grep -rn 'navruz' src/ returns nothing (case-insensitive).

• The footer and footer text show the project name from the DB.

Codex prompt: *Remove every hardcoded contact string from src/. Replace each with a read from getCurrentTenant(). Specifically: FloatingContact reads phoneNumber and telegramUrl from tenant; Footer reads project name and copyright string; NavbarClient reads phone display; ContactForm reads phone, telegram, sales-office address from tenant; ProjectTopView fallback image becomes tenant.masterPlanImage (or empty state if null); src/lib/project-copy.ts is deleted and src/lib/translations.ts's getTranslation helper handles*

tenant name/description/address from the new tenant fields (with the nameTranslations / descriptionTranslations / addressTranslations JSON columns that already exist on Project). Delete src/lib/project-copy.ts. PROJECT_EXPECTED_YEAR usages become tenant.expectedYear. After: grep -rn 'wxusan\|navruz\|+998 77 041' src/ must return zero matches.

Stage 4 — Dead-code cleanup

Estimated time: about 2 hours. Pure deletes. Lower bundle size, simpler onboarding for a future engineer, no ambiguity about which component is canonical. All these have zero importers.

Step 4.1 — Delete orphan components

Why: Seven components have zero importers and are dragging maintenance burden.

Files to touch:

```
src/components/IntentPopup.tsx (delete)
src/components/BuildingSelector.tsx (delete)
src/components/FloorPlanPolygon.tsx (delete)
src/components/FloorPlanSVG.tsx (delete)
src/components/FloorSelector.tsx (delete)
src/components/UnitDetailModal.tsx (delete)
src/components/GroupedApartmentModal.tsx (delete)
```

What to do:

• Confirm each has zero importers (grep -rn ComponentName src/ should show only the file itself).

• Delete the files.

• npm run build still passes.

Done when:

• Bundle size drops slightly.

• Build passes.

Codex prompt: *Verify each of these components has zero importers, then delete: src/components/IntentPopup.tsx, BuildingSelector.tsx, FloorPlanPolygon.tsx, FloorPlanSVG.tsx, FloorSelector.tsx, UnitDetailModal.tsx, GroupedApartmentModal.tsx. Run npm run build after to confirm nothing breaks.*

Step 4.2 — Delete duplicate public routes

Why: /vizual is a duplicate of the / explore section. /projects/[id] and /projects/[id]/explore are leftover scaffolding with the wrong design system and hardcoded English.

Files to touch:

```
src/app/vizual/ (entire directory, delete)
src/app/projects/ (entire directory, delete)
next.config.mjs (add redirects)
```

What to do:

• Delete src/app/vizual/.

• Delete src/app/projects/.

• Add redirects() in next.config.mjs: /vizual -> /, /projects/:id -> /, /projects/:id/explore -> /.

Done when:

• Hitting /vizual redirects to /.

• Hitting /projects/anything redirects to /.

• npm run build still passes.

Codex prompt: *Delete the entire src/app/vizual/ directory and the entire src/app/projects/ directory. Add a redirects() async function to next.config.mjs (alongside rewrites and headers) that 301-redirects /vizual to /, /projects/:projectId to /, and /projects/:projectId/explore to /. Run npm run build to confirm.*

Step 4.3 — Move puppeteer to devDependencies and trim Tailwind palette

Why: Puppeteer in production dependencies bloats the Vercel function. Tailwind colors navy/gold/status are defined but unused.

Files to touch:

```
package.json
tailwind.config.ts
```

What to do:

• Confirm puppeteer is not imported anywhere in src/: grep -rn 'from .puppeteer' src/ should return nothing.

• Move puppeteer from dependencies to devDependencies (or remove entirely if no script uses it).

• Remove the unused 'navy', 'gold', 'status' color groups from tailwind.config.ts unless grep finds usage.

• Run npm install and npm run build.

Done when:

• package.json shows puppeteer under devDependencies (or removed).

• Build passes.

• Vercel function size shrinks measurably.

Codex prompt: *First check whether puppeteer is imported anywhere in src/ via grep. If not, move it from dependencies to devDependencies in package.json. Then audit tailwind.config.ts: navy, gold, and status color groups are defined — check if any are used in src/ (grep for 'bg-navy', 'text-gold', 'bg-status'). Remove any unused color groups. Run npm install and npm run build.*

Step 4.4 — Archive old audit + clean repo root

Why: Old audit doc, scattered presentation folders.

Files to touch:

```
AUDIT_REPORT.md (move to docs/audits/2026-04.md)
docs/audits/ (new)
pentagi/ (review with owner)
presentation/ (review with owner)
```

What to do:

• mkdir docs/audits and move AUDIT_REPORT.md there as 2026-04.md.

• Ask the owner before deleting pentagi/ and presentation/.

Done when:

• Repo root contains only README.md, package.json, configs, and source dirs.

Codex prompt: *Create docs/audits/ and move the existing AUDIT_REPORT.md there renamed as 2026-04.md. Don't delete the pentagi/ or presentation/ directories yet — flag them for the owner to confirm.*

Stage 5 — Sales conversion improvements

Estimated time: about 1 day. Highest leverage per hour. Waitlist for sold/reserved units recovers 'almost-closed' leads. Hero price + completion year answer the buyer's first question in 3 seconds. Sticky mobile reserve bar removes the floor-plan scroll-trap on phones.

Step 5.1 — Add waitlist + similar-unit suggestions for unavailable units

Why: Today a buyer who falls in love with a sold unit hits a dead-end paragraph. Single highest-leverage conversion change.

Files to touch:

```
src/components/ApartmentLeadModal.tsx
src/components/SimilarUnits.tsx (new)
src/app/api/leads/route.ts (validSources update)
```

What to do:

• When status is 'reserved' or 'sold', replace the static paragraph with two sections:

• 1) A 'Notify me if it frees up' form using the existing name + phone fields, posting to /api/leads with source='waitlist'.

• 2) A 'Similar layouts available' horizontal scroll of up to 4 available units with the same rooms and area ± 5m². Click any tile opens its modal.

• Add 'waitlist' to the validSources allowlist in /api/leads.

• Track PostHog event 'Joined Waitlist' with apartment_number, rooms, area.

Done when:

• Opening a sold unit shows the waitlist form and similar-units scroller.

• Submitting the form creates a lead with source='waitlist'.

• Similar units shown all have status='available' and rooms == sold unit rooms.

Codex prompt: *In src/components/ApartmentLeadModal.tsx, replace the static 'unavailable' paragraph (in the else branch where isAvailable is false) with two new sections: (1) a 'Notify me when similar units become available' form that reuses the name/phone state and submits to /api/leads with source='waitlist'; (2) a horizontal scroller of up to 4 available units with the same rooms and area within ±5m² of the current unit. Create src/components/SimilarUnits.tsx that accepts the current unit + a list of all units and renders the tiles. Add 'waitlist' to validSources in /api/leads. Track 'Joined Waitlist' in PostHog with the unit metadata.*

Step 5.2 — Show price-from + completion year + availability count above the fold

Why: Premium buyers want a price range within 3 seconds. Today the hero has no price.

Files to touch:

```
src/components/ProjectTopView.tsx
src/app/page.tsx
```

What to do:

• Compute on the server: minPricePerM2 across available units, expectedYear (from tenant or PROJECT_EXPECTED_YEAR), available count, total count.

• Add a single subtle line under the hero title: '$X,XXX / m² dan · {available} of {total} available · ready Q{expectedYear}'.

• Localize all three labels.

Done when:

• Homepage hero shows starting price, availability ratio, and completion year.

Codex prompt: *In src/app/page.tsx, compute minPricePerM2 = Math.min(...availableUnits.filter(u => u.pricePerM2 || floorBase).map(u => u.pricePerM2 ?? floorBase)), availableCount, totalCount, expectedYear. Pass them as new props to ProjectTopView. In ProjectTopView, render a single subtle line below the heroTitle in the format 'from {price} / m² · {available} of {total} available · ready {expectedYear}', localized via next-intl in all three languages.*

Step 5.3 — Sticky mobile 'Reserve' bar on FloorPlanView

Why: On mobile, the reserve CTA is buried below the 80vh floor plan image. Sticky bar reduces friction to one tap.

Files to touch:

```
src/components/FloorPlanView.tsx
```

What to do:

• When a unit is selected and viewport width is < 1280px, render a position:fixed bottom bar containing: unit display number + area + price, a primary 'Reserve' button (opens ApartmentLeadModal), and a secondary Telegram icon button.

• Hide the bar when no unit is selected or when the lead modal is open.

• Add 80px bottom padding to the main content so the sticky bar doesn't cover the footer/dock.

Done when:

• On mobile, tapping a polygon shows a sticky bar at the bottom with the unit info and Reserve button.

• Tapping the bar opens the lead modal.

• On desktop (xl breakpoint), the bar does not render.

Codex prompt: *In src/components/FloorPlanView.tsx, when selectedUnit is set, render a position:fixed bottom-0 bar visible only below xl breakpoint. Bar shows the unit display number, area, price, a primary Reserve button (opens the existing ApartmentLeadModal via setLeadUnit), and a secondary Telegram icon link (use tenant.telegramUrl if step 3.3 done, otherwise fall back to existing constant). Hide the bar when leadUnit is set. Add 80px bottom padding to the main panel so it doesn't get covered.*

Step 5.4 — Add response-time SLA copy to lead forms

Why: Specific numbers reduce anxiety. 'Tez orada' is too vague for a buyer comparing 4 developers in one evening.

Files to touch:

```
src/components/ContactForm.tsx
src/components/ApartmentLeadModal.tsx
messages/uz.json, messages/ru.json, messages/en.json
```

What to do:

• Add a translation key contact.responseSLA = 'Our sales manager will call within 15 minutes (9:00–19:00).' in all three locales.

• Render it as small text below the submit button in both forms.

• After success state, add a 'Skip the wait — message us on Telegram' link.

Done when:

• Lead forms show an explicit SLA below the submit button.

• Success state offers a Telegram shortcut.

Codex prompt: *Add translation key contact.responseSLA in messages/uz.json, ru.json, en.json with text like 'Our sales manager will call within 15 minutes (9:00–19:00)' (localized). In ContactForm.tsx and ApartmentLeadModal.tsx, render this as a small caption below the submit button. After form success, add a secondary 'Skip the wait — message us on Telegram' link using tenant.telegramUrl (or the existing constant if step 3.3 not done).*

Step 5.5 — Wire Cloudinary helpers on every sketch image

Why: Today most sketch images render at full Cloudinary resolution but display at 96px. Heavy mobile payload.

Files to touch:

```
src/components/FloorPlanView.tsx
src/components/ApartmentLeadModal.tsx
src/components/FeaturedApartments.tsx
src/components/GroupedApartmentCard.tsx
src/app/kvartiralar/ApartmentsClient.tsx
```

What to do:

• Find every usage.

• Replace src with getCardImageUrl(...) for tile contexts and getFullImageUrl(...) for modal-main contexts. Use getThumbnailUrl(...) for 64–96px thumbnails.

• Run npm run build and visually verify the homepage and floor-plan view.

Done when:

• All sketch image URLs include the Cloudinary transform parameters in the rendered HTML (visible in dev-tools network).

• Mobile image payload drops noticeably.

Codex prompt: *Audit every tag in FloorPlanView, ApartmentLeadModal, FeaturedApartments, GroupedApartmentCard, ApartmentsClient where src comes from unit.sketchImage[2-4]. Replace with the appropriate helper from src/lib/cloudinary: getThumbnailUrl for ≤96px, getCardImageUrl for tiles ≤600px, getFullImageUrl for modal mains. Build and confirm.*

Step 5.6 — Auto-scroll the selected unit panel into view on mobile

Why: On phones, tapping a polygon at the top of the floor plan leaves the selection card offscreen.

Files to touch:

```
src/components/FloorPlanView.tsx
```

What to do:

• Add a ref to the selected-unit panel.

• useEffect on selectedUnit?.id: if window.innerWidth < 1280, panelRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' }).

Done when:

• On mobile, tapping a polygon scrolls the page so the new selection card is visible.

Codex prompt: *In src/components/FloorPlanView.tsx, add a useRef on the selected-unit panel div. Add a useEffect with dependency [selectedUnit?.id]: if selectedUnit and window.innerWidth < 1280, call panelRef.current?.scrollIntoView({ behavior: 'smooth', block: 'start' }) inside a requestAnimationFrame.*

Stage 6 — Codebase quality and type safety

Estimated time: about 1 day. Pay down the technical debt the audit flagged: duplicated status colors, server-computed display numbers, admin translations, Lead status enum, /api/units pagination, Zod validation. Each is small.

Step 6.1 — One shared status-style module

Why: Three components currently define status colors with different palettes.

Files to touch:

```
src/lib/status-style.ts (new)
src/components/FloorPlanView.tsx
src/components/FloorPlanPolygon.tsx (already deleted in 4.1)
src/lib/utils.ts (remove duplicate)
```

What to do:

• Create src/lib/status-style.ts exporting getStatusMeta(status, active?) returning { label, fillColor, strokeColor, badgeClass, textColor }.

• Replace inline getStatusMeta in FloorPlanView with the imported one.

• Remove getStatusColor and getStatusBg from src/lib/utils.ts; update callers.

Done when:

• Single source of truth for status palette.

• Build passes.

Codex prompt: *Create src/lib/status-style.ts exporting getStatusMeta(status: string, active: boolean = false) that returns { label: string (localized), fillColor, strokeColor, badgeClass, textColor } for 'available', 'reserved', 'sold'. Use the clay/amber/red palette from the current FloorPlanView component. Then replace the inline getStatusMeta function in FloorPlanView with the shared one. Remove getStatusColor and getStatusBg from src/lib/utils.ts and update any callers.*

Step 6.2 — Server-computed displayNumber

Why: Five components recompute the floor-prefix logic with the same regex; collisions on floors with >99 units.

Files to touch:

```
src/lib/public-selects.ts
src/lib/unit-display.ts (new)
src/components/FloorPlanView.tsx, ApartmentLeadModal.tsx, FeaturedApartments.tsx,
GroupedApartmentCard.tsx, kvartiralar/ApartmentsClient.tsx
```

What to do:

• Create src/lib/unit-display.ts with computeDisplayNumber(unitNumber, floorNumber).

• Compute displayNumber on the server in publicProjectSelect post-processing (a thin wrapper that maps over units before sending to client).

• Refactor every callsite that imports the regex helper to use unit.displayNumber.

Done when:

• grep -rn 'padStart(2' src/components/ shows no remaining duplicates.

• Units render the same display number everywhere.

Codex prompt: *Create src/lib/unit-display.ts exporting computeDisplayNumber(unitNumber: string, floorNumber: number). Then wherever the server fetches projects/floors/units for public consumption (page.tsx, kvartiralar/page.tsx, ExploreClient input), map over units to attach a computed displayNumber field. Update the TypeScript Unit interface used by client components to include displayNumber. Replace every*

getDisplayNumber/regex-based callsite in the 5 listed components to use unit.displayNumber.

Step 6.3 — Translate the admin labels

Why: Russian-speaking admins (likely half the real users) get a half-translated UI.

Files to touch:

```
messages/uz.json, messages/ru.json, messages/en.json
src/app/portal/management-x7k9/page.tsx
src/app/portal/management-x7k9/leads/LeadsClient.tsx
src/components/AdminSidebar.tsx
src/app/portal/management-x7k9/projects/ProjectsClient.tsx
src/app/portal/management-x7k9/projects/[projectId]/buildings/BuildingsClient.tsx
```

What to do:

• Audit every hardcoded English string in the admin tree.

• Add admin.* translation keys for: 'Overview', 'Recent leads', 'Quick actions', 'Inventory', 'See all', 'View leads', 'Manage project', 'Buildings', 'Building images', 'Units', 'No leads yet', occupancy %, etc.

• Replace literals with t(key).

Done when:

• Switching admin locale to Russian translates all visible labels.

• No literal English string remains in admin server pages or clients (except brand 'UyJoy').

Codex prompt: *Audit src/app/portal/management-x7k9/ for hardcoded English strings (the dashboard, leads page, sidebar, projects/buildings clients). Add the missing keys to messages/uz.json, ru.json, en.json under the 'admin' namespace. Replace every literal with t('admin.key'). Confirm by switching locale via the AdminSidebar selector and seeing the labels switch.*

Step 6.4 — Lead status enum

Why: Status is freetext in DB with three sources of truth for valid values.

Files to touch:

```
prisma/schema.prisma
src/lib/lead-status.ts (new)
src/app/api/leads/[id]/route.ts
src/app/portal/management-x7k9/leads/LeadsClient.tsx
src/app/portal/management-x7k9/page.tsx
```

What to do:

• Option A (recommended): add Prisma enum LeadStatus { new inCRM callback inProgress contacted converted notInterested closed }.

• Option B (lower-risk): keep String but create src/lib/lead-status.ts with the const + zod schema; PUT /api/leads/[id] validates against it.

• Pick A or B. If A, write a migration that maps existing values then ALTERs to enum.

• LeadsClient, dashboard, and any other reader import the single constant.

Done when:

• PUT /api/leads/[id] with status='bogus' returns 400.

• Single source of truth in src/lib/lead-status.ts.

Codex prompt: *Create src/lib/lead-status.ts exporting LEAD_STATUSES (readonly tuple) and LeadStatusSchema (zod enum). Add validation in /api/leads/[id] PUT to reject any status not in the set. Update LeadsClient and the admin dashboard to import LEAD_STATUSES from this module. If you also want to enforce at the DB level, add a Prisma enum LeadStatus and migrate — but make sure to map every existing*

string value first. Otherwise leave the column as String and rely on the API validation.

Step 6.5 — Default-paginate /api/units

Why: Right now without page param, the endpoint returns all units in one response.

Files to touch:

```
src/app/api/units/route.ts
src/app/portal/management-x7k9/projects/[projectId]/units/UnitsClient.tsx
```

What to do:

• Default to pagination when page param is absent: page=1, limit=50.

• Keep the optional unpaginated path behind a ?all=true admin-only flag, gated by middleware (or removed entirely).

• Update UnitsClient to request pages and render a 'load more' button.

Done when:

• GET /api/units returns at most 50 units by default.

• Admin units page still works.

Codex prompt: *In /api/units route.ts GET, default to paginated response (page=1, limit=50) when no page param is provided. Add ?all=true as an admin-only escape hatch that returns everything (rely on middleware to require auth, since this fetches sensitive customer data inside the admin Unit type). Update UnitsClient to fetch by page and render a 'Load more' button, or keep ?all=true if pagination would be too invasive — pick the lower-effort path. Either way, anonymous GET /api/units must return at most 50 units.*

Step 6.6 — Zod validation on every API route

Why: Most routes accept body.foo with no shape checking. Garbage data lands in JSON columns.

Files to touch:

```
src/lib/schemas/*.ts (new)
every src/app/api/**/route.ts
```

What to do:

• Install zod (already a dep via prisma — confirm; if not, npm i zod).

• Create src/lib/schemas/lead.ts, project.ts, building.ts, floor.ts, unit.ts, hero-image.ts, faq.ts, user.ts each exporting a CreateSchema and an UpdateSchema.

• In every route's POST/PUT, parse body with the schema; return 400 with .flatten() on failure.

• polygonData specifically: z.array(z.object({ x: z.number().min(0).max(100), y: z.number().min(0).max(100) })).min(3).

Done when:

• POST /api/buildings with polygonData: 'not-an-array' returns 400 instead of writing garbage to DB.

• All routes return structured 400 errors for bad shapes.

Codex prompt: *Install zod if missing. Create src/lib/schemas/ with one file per resource (lead, project, building, floor, unit, hero-image, faq, user) exporting CreateSchema and UpdateSchema. In every API route POST/PUT, parse the body with the schema and return NextResponse.json({ error: 'Invalid input', details: parsed.error.flatten() }, { status: 400 }) on failure. For polygonData, schema is z.array(z.object({ x: z.number().min(0).max(100), y: z.number().min(0).max(100) })).min(3). Numbers like pricePerM2 must be z.number().nonnegative().*

Stage 7 — Pre-launch verification

Estimated time: about 2 hours. Final smoke test before the site touches a paying client. Run every check; fix anything that fails.

Step 7.1 — Build and bundle audit

Why: Confirm the production build is clean and lean.

Files to touch:

```
(commands only)
```

What to do:

• Run: npm run build.

• Confirm no TS errors, no warnings other than known ones.

• Inspect .next/analyze if @next/bundle-analyzer is configured, otherwise eyeball .next/static/chunks/ for chunks >300KB.

• Confirm maplibre-gl ships only with /#location, not the main bundle (dynamic import).

• Confirm puppeteer is NOT in the production node_modules (after step 4.3).

Done when:

• Build passes.

• Main page first-load JS < 300 KB ideally, < 400 KB acceptable.

• No puppeteer in production deps.

Codex prompt: *Run npm run build. Report any errors or warnings. Show the bundle sizes from the build output. If MapLibre is in the main chunk (visible in the chunk listing), dynamically import LocationInfrastructure so it loads only when scrolled to. Confirm puppeteer is not in production dependencies.*

Step 7.2 — Lighthouse and Core Web Vitals

Why: Premium real-estate site needs to feel premium-fast.

Files to touch:

```
(no files — Lighthouse run only)
```

What to do:

• Run Lighthouse on production build (npm run start, hit localhost:3000) in incognito.

• Run on /, /kvartiralar, and one floor-explore deep link (/?building=xxx&floor;=yyy).

• Targets: Performance ≥ 85 mobile, ≥ 95 desktop. LCP < 2.5s. CLS < 0.1.

• Fix anything red. Most common offenders: large hero images (use getHeroImageUrl), late MapLibre.

Done when:

• All three URLs hit performance targets on mobile.

Codex prompt: *Run npm run build then npm run start. Run Lighthouse in Chrome incognito on three URLs: /, /kvartiralar, /?building=&floor;= (use any valid IDs from the DB). Report Performance, LCP, CLS for each. If anything is below 85 on mobile or LCP > 2.5s, identify the offender (usually a Cloudinary image not using getHeroImageUrl, or a synchronously imported MapLibre).*

Step 7.3 — End-to-end smoke test

Why: Run the full funnel manually.

Files to touch:

```
(manual test)
```

What to do:

• Anonymous flow: open /, click hero polygon, drill to a unit, submit lead form. Confirm lead appears in admin and Sentry stays quiet.

• Admin flow: log in, create a building, add a floor, upload a floor plan, draw a polygon, save a unit, mark unit reserved with customer name + phone.

• Refresh public site within 5s and confirm the unit shows reserved (validates stage 2.1 invalidation).

• Sold-unit waitlist: open a sold unit on /, submit the waitlist form, confirm it lands in admin with source='waitlist' (validates 5.1).

• CSV export: download leads CSV, open in Excel, confirm no formula-prefixed cells executed.

• Tenant test (if stage 3 done): change a tenant setting (phone), confirm it appears in floating contact + footer within 60s.

Done when:

• All six flows complete without errors.

Codex prompt: *Run a full manual smoke test as a real user would. Step through: (1) submit a lead from the homepage contact form; (2) submit a lead from an apartment modal; (3) admin login, create+edit+delete a building/floor/unit; (4) mark a unit as reserved with customer name/phone, verify it goes private to public users; (5) wait <5s, refresh public site, confirm cache invalidation works; (6) submit a waitlist lead from a sold unit; (7) export leads CSV and open in Excel, confirm no formula injection. Report which steps pass and fail.*

Step 7.4 — Security verification

Why: Confirm the hardening from the first batch + stage 1 is still live.

Files to touch:

```
(scan only)
```

What to do:

• view-source on / and /kvartiralar: grep for 'customerPhone', 'customerName', 'customerNotes' — must be zero.

• curl -sI https://staging | grep -i 'strict-transport\|x-frame\|content-security' — confirm all four security headers present.

• Try GET /api/leads as anonymous — must return 401.

• Try POST /api/leads 20 times in 1 minute from one IP — should rate-limit by request 6.

• Try POST /api/upload as anonymous — must return 401.

• Try POST /api/ai/detect-floors with imageUrl='/uploads/../../../etc/passwd' — must return 400.

Done when:

• All six checks pass.

Codex prompt: *Verify the security hardening is intact on staging: (1) curl view-source of / and /kvartiralar, grep for customerPhone/customerName/customerNotes — must be zero matches. (2) curl -I against staging and confirm Strict-Transport-Security, X-Frame-Options, X-Content-Type-Options, Content-Security-Policy headers present. (3) Anonymous GET /api/leads returns 401. (4) POST /api/leads 20× in 60s from one IP gets rate-limited. (5) Anonymous POST /api/upload returns 401. (6) POST /api/ai/detect-floors with imageUrl '/uploads/../../etc/passwd' returns 400. Report which checks pass.*

Step 7.5 — Backups and rollback

Why: Before any production traffic, confirm we can recover.

Files to touch:

```
(infrastructure)
```

What to do:

• Enable automated daily Postgres backups (Neon: Point-in-Time Recovery; or pg_dump cron on a managed instance).
• Take a manual snapshot named 'pre-v2-launch' immediately before the deploy.
• Confirm Vercel keeps the previous deploy promotable in case of rollback.
• Document the rollback procedure in docs/runbook.md: revert deploy via Vercel dashboard, restore DB via Neon PITR.

Done when:

• Daily backups enabled.
• Pre-launch snapshot taken.
• Rollback procedure documented in repo.

Codex prompt: *Enable Neon Point-in-Time Recovery (or whatever your Postgres host's equivalent is). Take a manual snapshot labeled 'pre-v2-launch' right before the production deploy. Confirm in Vercel that the previous deploy is preserved and can be promoted as a rollback target. Create docs/runbook.md describing the rollback procedure in 5 lines: how to revert the Vercel deploy, how to PITR the database, who to notify.*

Order of operations summary

Run Stage 1 this week (compliance + Sentry + Turnstile). Run Stage 2 the same week or next (data integrity, indexes — these prevent admin trust collapse). Run Stage 3 only if you're selling multi-tenant from a single deployment. Run Stages 4–6 in parallel by topic during a polish week. Run Stage 7 the morning of launch.

If you must cut something: skip Stage 3 (sell to one client first, retro-fit multi-tenancy when you have a second client signed). Skip Stage 6.6 (Zod). Skip Stage 5.6 (auto-scroll). **Do not skip Stage 1, Stage 2, Stage 4.2 (delete leftover pages), Stage 7.**

Status header for codex sessions

When you hand a step to codex, paste this preamble before the codex prompt:

'You are working on UyJoy, a Next.js 14 App Router app with Prisma/Postgres, NextAuth credentials, Cloudinary, deployed on Vercel. The codebase is at /Users/xusan/Projects/uy-joy. Read the relevant files first, then make the smallest correct change. Run npx tsc --noEmit and npm run build before reporting done. Don't introduce new dependencies without naming them up front.'